

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ «Национальный исследовательский университет ИТМО»
(Университет ИТМО)

Факультет прикладной информатики
Образовательная программа Мобильные и сетевые технологии
Направление подготовки 09.03.03 Прикладная информатика

О Т Ч Е Т
Отчёт по курсовой работе дисциплины Мобильная
разработка (Android и IOs)

Обучающийся:
Оспельников Алексей Владимирович, К3440

Преподаватель:
Шатинский Григорий Сергеевич

Санкт-Петербург
2026

АННОТАЦИЯ

Курсовая работа посвящена разработке учебного Android-приложения, имитирующего базовый функционал мессенджера. Пользователь может просматривать ленту сообщений, редактировать профиль и изменять настройки приложения. Разработка велась поэтапно в рамках четырех лабораторных работ. На первом этапе был создан каркас интерфейса и реализована навигация между экранами. На втором этапе внедрена архитектура MVVM с управлением состоянием экранов через ViewModel и LiveData. На третьем этапе добавлена загрузка данных из публичного REST-API и сохранение сообщений в локальную базу данных Room, что обеспечило возможность офлайн-доступа. На заключительном этапе усовершенствован пользовательский интерфейс, реализованы лайки, фоновая синхронизация данных через WorkManager и система уведомлений.

В работе использованы язык Kotlin, среда разработки Android Studio, компоненты Android Jetpack (ViewModel, LiveData, Navigation, Room, WorkManager), а также библиотеки Retrofit и OkHttp для работы с сетью. Интерфейс приложения построен с использованием элементов Material Design. Разработанное приложение демонстрирует полный цикл работы с данными в мобильном клиенте: получение сообщений из сети, их кэширование в базе данных, отображение в списке и фоновое обновление с уведомлением пользователя.

СОДЕРЖАНИЕ

1 ОБЩАЯ ХАРАКТЕРИСТИКА ПРОЕКТА.....	5
1.1 Назначение и функции приложения.....	5
1.2 Этапы выполнения.....	5
2 ТЕХНОЛОГИЧЕСКАЯ БАЗА ПРОЕКТА.....	7
2.1 Платформа и инструменты разработки.....	7
2.2 Архитектура MVVM.....	7
2.3 Используемые библиотеки.....	7
3 РЕАЛИЗАЦИЯ ЛАБОРАТОРНЫХ РАБОТ.....	9
3.1 Лабораторная работа №1. Интерфейс и навигация.....	9
3.2 Лабораторная работа №2. Управление состоянием через ViewModel.....	11
3.3 Лабораторная работа №3. Сеть и база данных.....	12
3.4 Лабораторная работа №4. Интерфейс, фоновые задачи и уведомления...	13
4 ПОЛЬЗОВАТЕЛЬСКИЙ ИНТЕРФЕЙС.....	16
4.1 Лента сообщений.....	17
5 ТЕСТИРОВАНИЕ.....	19

ВВЕДЕНИЕ

Мессенджеры относятся к числу наиболее востребованных мобильных приложений. Пользователи ожидают от них интуитивно понятного интерфейса, стабильной работы при нестабильном соединении и своевременных уведомлений. Поэтому даже учебный проект в формате «мессенджера» является удобной платформой для отработки ключевых навыков Android-разработки: организации навигации, управления состоянием экранов, работы с сетью и локальным хранилищем, а также реализации фоновых задач.

Целью курсовой работы является разработка учебного Android-приложения мессенджера с использованием современного стека технологий и архитектурного подхода MVVM.

Для достижения цели решались следующие задачи. Сначала необходимо было спроектировать основной интерфейс приложения и систему навигации между экранами. Затем выполнялась интеграция ViewModel и LiveData для сохранения состояния профиля и настроек при повороте экрана. Следующим этапом стала работа с публичным REST API и сохранение полученных данных в базе Room, что обеспечило возможность офлайн-доступа. После этого реализовывалось фоновое обновление данных через WorkManager и уведомления о новых сообщениях. Завершающей задачей была настройка визуального оформления интерфейса с использованием компонентов Material Design, включая поддержку светлой и тёмной темы.

В отчете рассмотрены используемые технологии, структура проекта, реализация каждого этапа и результаты тестирования.

1 ОБЩАЯ ХАРАКТЕРИСТИКА ПРОЕКТА

1.1 Назначение и функции приложения

Разработанное приложение представляет собой учебный мессенджер. Пользователь может просматривать список сообщений, условно представляющих диалоги или новости. Каждое сообщение можно отметить как понравившееся. Кроме того, реализованы отдельные экраны профиля, где редактируются имя и статус пользователя, и настроек, где осуществляется переключение темы оформления.

Загрузка сообщений выполняется из открытого сервиса JSONPlaceholder, который возвращает тестовые данные в формате JSON. Это позволяет продемонстрировать работу с сетью без необходимости создавать собственный сервер. Для устойчивой работы при отсутствии интернет-соединения сообщения сохраняются в локальной базе данных Room и подгружаются из нее, когда обращение к сети невозможно.

1.2 Этапы выполнения

Работа разбита на четыре логических этапа, соответствующих лабораторным заданиям.

На первом этапе был создан каркас приложения: одна активность, несколько фрагментов и навигация с использованием BottomNavigationView и Navigation Component.

Во втором этапе добавлены классы ViewModel и реализовано реактивное обновление интерфейса через LiveData, что позволило организовать сохранение и управление состоянием профиля и настроек при изменении конфигурации (например, поворот экрана).

Третий этап посвящен интеграции с REST-API и библиотекой Room, а также реализации офлайн-режима для работы с сохраненными данными при отсутствии интернет-соединения.

Четвертая лабораторная работа расширяет функционал приложения: реализован кастомный адаптер для списка сообщений, возможность ставить

лайки, фоновая синхронизация данных через WorkManager, система уведомлений и запрос необходимых разрешений при первом запуске.

2 ТЕХНОЛОГИЧЕСКАЯ БАЗА ПРОЕКТА

2.1 Платформа и инструменты разработки

Приложение ориентировано на платформу Android с минимальной версией API 24 (Android 7.0). В качестве языка программирования выбран Kotlin, который официально рекомендован Google для разработки под Android. Разработка велась в Android Studio, сборка проекта осуществлялась с помощью системы Gradle и файла build.gradle.kts, в котором подключены все необходимые зависимости и библиотеки.

2.2 Архитектура MVVM

В качестве архитектурного подхода применён паттерн Model-View-ViewModel (MVVM). Представление (Activity и Fragment) отвечает только за отображение данных и обработку действий пользователя. ViewModel хранит состояние экранов, вызывает методы репозитория и не зависит от конкретных классов фрагментов. Слой данных обеспечивает доступ к сети и локальной базе. Такой подход позволяет изменять интерфейс без переписывания логики работы с данными, а код становится более структурированным и удобным для расширения.

Связь между View и ViewModel реализована через LiveData: фрагменты подписываются на объекты LiveData и автоматически реагируют на изменения данных.

2.3 Используемые библиотеки

В проекте применяются компоненты Android Jetpack: ViewModel и LiveData для управления состоянием экранов, Navigation Component для организации навигации, Room для локального хранения данных и WorkManager для выполнения фоновых задач. Для сетевого взаимодействия используется связка Retrofit и OkHttp, позволяющая описывать HTTP-запросы декларативно и выполнять их в корутинах. Пользовательский интерфейс построен на базе Material Components: применяются MaterialToolbar, FloatingActionButton, MaterialCardView,

BottomNavigationView и другие элементы, обеспечивающие современный дизайн и удобство взаимодействия.

3 РЕАЛИЗАЦИЯ ЛАБОРАТОРНЫХ РАБОТ

3.1 Лабораторная работа №1. Интерфейс и навигация

На первом этапе разработки сформирована базовая структура приложения. Создана главная активность MainActivity, в которой размещён NavHostFragment, обслуживающий переходы между фрагментами. Нижняя навигация реализована с помощью BottomNavigationView. В ресурсах определён файл menu_bottom_nav.xml, где перечислены пункты «Лента», «Профиль» и «Настройки» с соответствующими иконками.

Связывание панели навигации с контроллером навигации выполняется в MainActivity методом setupWithNavController, что обеспечивает автоматический переход к соответствующему фрагменту при выборе вкладки без необходимости писать дополнительный код для обработки нажатий.

Для анализа жизненного цикла фрагментов создан базовый класс LoggingFragment, в котором переопределены методы onCreate, onStart, onResume, onPause, onStop, onDestroyView и onDestroy. В каждом методе производится запись сообщений в Logcat, что позволило отследить последовательность вызовов при открытии и закрытии экранов, а также при повороте устройства.

Сразу был создан файл strings.xml в 2 экземплярах, чтобы была возможность пользоваться приложением для англоязычных пользователей.

На рисунке 1 представлен внешний вид главного экрана профиля.

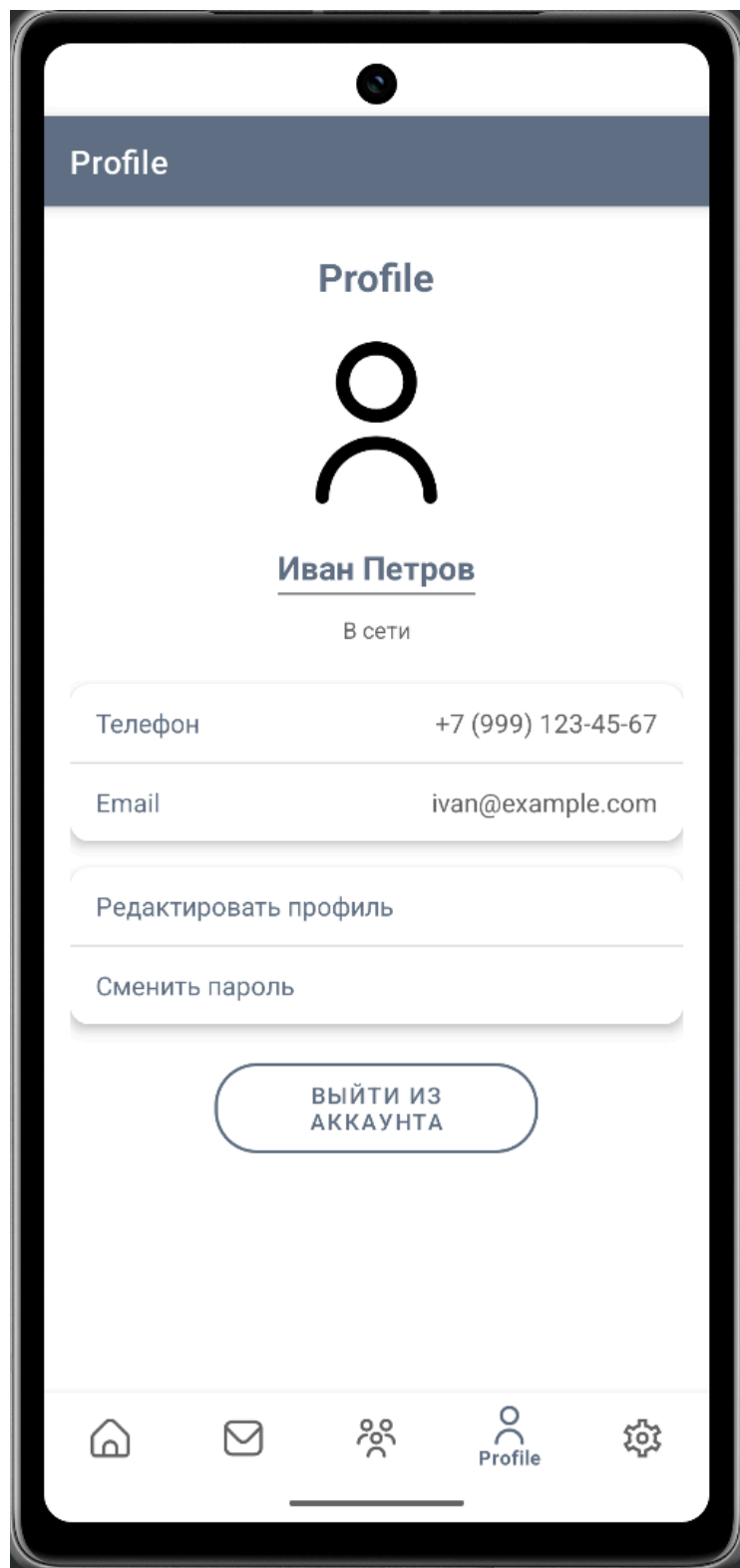


Рисунок 1 – Внешний вид экрана профиля

3.2 Лабораторная работа №2. Управление состоянием через ViewModel

Вторая лабораторная работа была посвящена внедрению архитектурного подхода Model–View–ViewModel (MVVM). Для экрана профиля создан класс ProfileViewModel, в котором с использованием MutableLiveData хранятся имя и статус пользователя.

Указанные данные инициализируются значениями по умолчанию, однако могут быть изменены пользователем в процессе работы приложения. В классе реализованы методы changeName и changeStatus, отвечающие за обновление соответствующих полей. При изменении значений ViewModel автоматически уведомляет представление, что обеспечивает реактивное обновление интерфейса без прямого обращения фрагмента к модели данных.

Фрагмент настроек SettingsFragment отвечает за переключение темы оформления приложения. Для доступа к элементам интерфейса используется ViewBinding (FragmentSettingsBinding). При создании представления выполняется чтение текущего режима темы через вспомогательный класс ThemeManager, после чего состояние переключателя синхронизируется с сохранённым значением.

Изменение темы реализовано через обработчик setOnCheckedChangeListener у компонента Switch. При изменении положения переключателя вызывается метод ThemeManager.setDarkMode, который сохраняет новое состояние и применяет соответствующий режим оформления. Таким образом, пользователь может в любой момент изменить тему приложения, а выбранный режим сохраняется между запусками.

Для анализа жизненного цикла фрагмента переопределены основные callback-методы (onCreate, onStart, onResume, onPause, onStop, onDestroyView, onDestroy) с выводом сообщений в Logcat. Это позволило наглядно отследить последовательность вызовов и убедиться в корректной работе навигации и пересоздания экранов.

В экране профиля состояние пользователя управляется через ViewModel, получаемую с помощью делегата `activityViewModels()`, что обеспечивает повторное использование одного экземпляра при пересоздании фрагмента. Обновление интерфейса происходит через наблюдателей LiveData, а передача пользовательского ввода обратно в ViewModel реализована с помощью слушателей `doOnTextChanged`. Такой подход обеспечивает двустороннюю связь между представлением и логикой без прямой зависимости фрагмента от модели данных.

В результате логика хранения состояния профиля вынесена во ViewModel, а настройки темы инкапсулированы в отдельном менеджере. При повороте устройства данные профиля сохраняются автоматически, а выбранная тема остаётся неизменной.

3.3 Лабораторная работа №3. Сеть и база данных

Третья лабораторная работа посвящена интеграции с удалённым REST-сервисом и организации локального хранения данных. В качестве тестового источника сообщений используется сервис JSONPlaceholder. Для сетевого взаимодействия применена библиотека Retrofit, которая позволяет описывать HTTP-запросы декларативно через интерфейс.

Описание API вынесено в интерфейс `MessageApi`. Основной метод `getMessages` выполняет GET-запрос и возвращает список объектов `MessageDto`, представляющих данные в том виде, в котором они приходят с сервера. Полученные DTO далее преобразуются в сущности базы данных.

Для локального хранения используется Room. Создана сущность `MessageEntity`, соответствующая таблице сообщений, а также `MessageDao`, содержащий методы вставки и чтения данных. База данных инициализируется через `Room.databaseBuilder`.

Логика работы с сетью и базой инкапсулирована в репозитории. Сначала выполняется запрос к API, после успешного ответа данные сохраняются в Room, после чего интерфейс получает сообщения уже из

локальной базы. Такой подход позволяет реализовать офлайн-режим: при отсутствии интернета приложение продолжает отображать ранее загруженные сообщения.

В ViewModel загрузка данных запускается в корутине, а результаты передаются во фрагмент через LiveData. Фрагмент подписывается на изменения и автоматически обновляет список при появлении новых записей.

Таким образом, на третьем этапе реализована связка Retrofit + Room с единым репозиторием, обеспечивающая синхронизацию удалённых данных с локальным хранилищем и устойчивую работу приложения без постоянного подключения к сети.

3.4 Лабораторная работа №4. Интерфейс, фоновые задачи и уведомления

Четвёртая лабораторная работа направлена на расширение функциональности приложения и приближение его к поведению реального мессенджера. На этом этапе реализованы отображение списка сообщений, система отметок «нравится», фоновое обновление данных и уведомления.

Для вывода сообщений используется RecyclerView с кастомным адаптером MessageAdapter. Каждый элемент списка представлен карточкой (MaterialCardView), внутри которой отображаются заголовок и текст сообщения, а также кнопка лайка. Адаптер получает список объектов из ViewModel и обновляет содержимое через DiffUtil, что позволяет эффективно перерисовывать только изменённые элементы.

Функциональность лайков реализована на уровне локальной базы данных. В сущность сообщения добавлено булево поле isLiked. При нажатии на кнопку соответствующее значение изменяется, после чего обновлённая запись сохраняется в Room. Так как интерфейс подписан на LiveData из базы данных, изменение статуса лайка сразу отражается в списке без дополнительной логики во фрагменте.

Для периодического обновления сообщений используется

WorkManager. Создан класс SyncWorker, который в фоновом режиме обращается к API, сохраняет новые данные в базу и завершает выполнение. Планировщик запускает задачу с заданным интервалом, что позволяет синхронизировать данные даже при свернутом приложении.

После успешной синхронизации формируется системное уведомление о появлении новых сообщений. Для этого настроен NotificationChannel, а сами уведомления создаются через NotificationCompat.Builder. При первом запуске приложения запрашивается разрешение на отправку уведомлений, что соответствует требованиям современных версий Android.

Дополнительно реализована обработка разрешений и базовая индикация состояний загрузки. Все фоновые операции выполняются вне главного потока, что обеспечивает стабильную работу интерфейса.

В результате четвёртого этапа приложение получило полноценный экран ленты с интерактивными элементами, поддержку фоновой синхронизации и уведомлений, что завершает формирование учебного мессенджера и демонстрирует применение ключевых компонентов Android Jetpack на практике.

Экран ленты с реализованной системой лайков показана на рисунке 2.



Рисунок 2 – Внешний вид ленты с лайками

4 ПОЛЬЗОВАТЕЛЬСКИЙ ИНТЕРФЕЙС

Пользовательский интерфейс приложения реализован с использованием компонентов Material Design и поддерживает два режима отображения: светлую и тёмную темы. Переключение режима применяется ко всему приложению и влияет на оформление всех экранов. Для этого используется переключатель темы в меню настроек, а выбранное состояние сохраняется во ViewModel, что обеспечивает корректное восстановление режима при пересоздании экранов и повороте устройства. Вид светлой и темной темы показан на рисунке 3 и 4 соответственно.

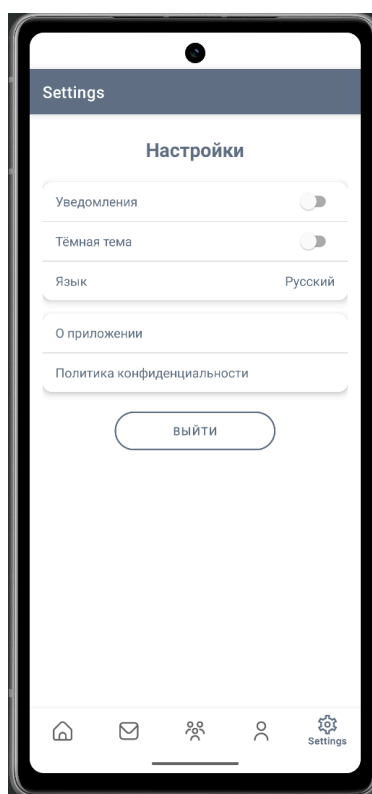


Рисунок 3 – Внешний вид светлой темы

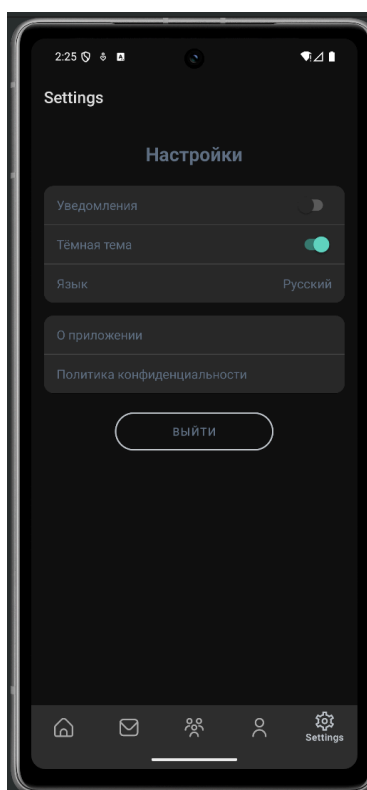


Рисунок 4 – Внешний вид темной темы

4.1 Лента сообщений

Главный экран приложения представляет собой ленту сообщений. В верхней части расположен MaterialToolbar с заголовком «Лента». Основную область экрана занимает список сообщений, реализованный с помощью RecyclerView.

Каждое сообщение отображается в виде карточки MaterialCardView: слева размещена иконка пользователя, рядом выводятся имя отправителя и текст сообщения, справа расположена кнопка лайка в виде сердца. Пользователь может отметить любое сообщение как понравившееся, при этом состояние сохраняется локально.

В правом нижнем углу экрана находится FloatingActionButton, предназначенная для ручного обновления данных. Во время загрузки сообщений поверх списка отображается индикатор ProgressBar. В случае ошибки загрузки пользователю выводится краткое уведомление в текстовом поле.

Нижняя панель навигации `BottomNavigationView` закреплена в основной активности и обеспечивает быстрый переход между экранами ленты, профиля и настроек.

Итоговый вид ленты сообщений показан на рисунке 5.

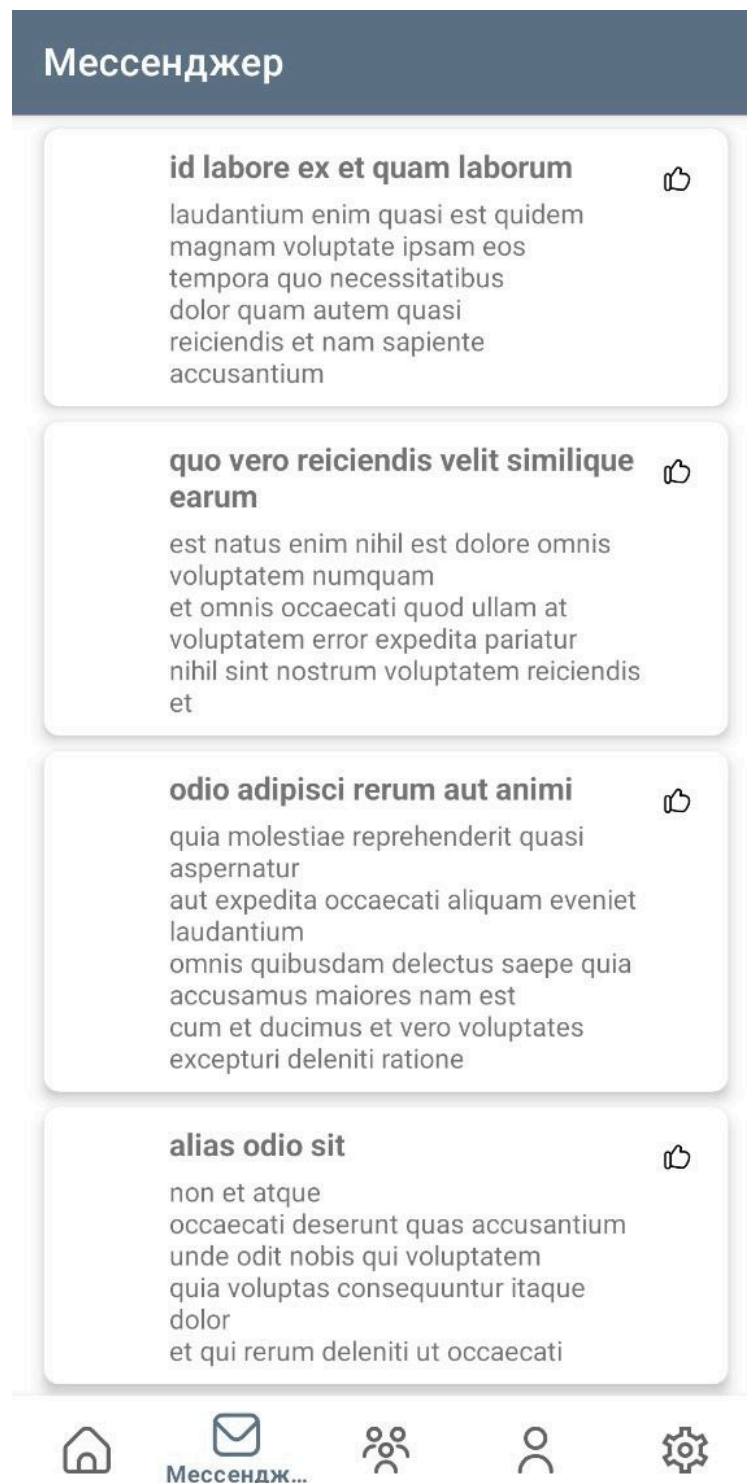


Рисунок 5 – Лента сообщений

5 ТЕСТИРОВАНИЕ

Работоспособность приложения проверялась вручную как на Android-эмуляторе, так и на физическом устройстве. В процессе тестирования основное внимание уделялось корректности навигации, сохранению состояния экранов, работе с сетью и локальной базой данных.

В первую очередь была проверена навигация между разделами, реализованная через нижнюю панель BottomNavigationView. Переходы между вкладками «Лента», «Профиль» и «Настройки» выполнялись корректно, без сбоев. При возврате на ранее открытые экраны их состояние сохранялось.

Далее тестировалась работа экрана профиля. Изменение имени и статуса пользователя сопровождалось поворотом устройства, после чего введённые данные сохранялись. Это подтверждает корректное использование ViewModel для хранения состояния.

Также была проверена работа фоновой синхронизации через WorkManager. Воркер выполнялся по расписанию и после успешного обновления отображал системное уведомление. На устройствах с Android 13 и выше уведомления появлялись только после подтверждения соответствующего разрешения пользователем, что соответствует требованиям платформы.

В ходе тестирования критических ошибок, приводящих к аварийному завершению приложения, выявлено не было. Отдельные сетевые ошибки и таймауты корректно обрабатывались и отображались пользователю в виде кратких уведомлений.

ЗАКЛЮЧЕНИЕ

В рамках курсовой работы разработано учебное Android-приложение, демонстрирующее современные подходы к мобильной разработке. Реализованы три основных экрана, единая навигация, архитектура MVVM, загрузка данных из REST API с кэшированием в локальной базе Room, фоновая синхронизация через WorkManager и система уведомлений. Интерфейс построен с использованием компонентов Material Design и поддерживает переключение темы.

Созданная архитектура облегчает дальнейшее развитие проекта. Возможны расширения, включая авторизацию, реальную отправку сообщений, поиск и пагинацию в ленте, пуш-уведомления через Firebase Cloud Messaging, модульное тестирование ViewModel и репозитория, а также настройку непрерывной интеграции.